

Implementação do Analisador Léxico (Lexer)

Abaixo está o código-fonte em Python responsável pela tokenização do nosso projeto. Ele utiliza expressões regulares (`re`) para converter a string bruta em uma lista de tokens estruturados.

```
1 import re
2
3 # =====
4 # ANALISADOR LEXICO (LEXER)
5 # =====
6
7 TOKEN_SPECIFICATION = [
8     ('NUMBER', r'\d+(\.\d+)?'), # Numeros inteiros ou
    decimais
9     ('IDENT', r'[A-Za-z_][A-Za-z0-9_]*'), #
    Identificadores/variaveis
10    ('POW', r'\*\*|\^'), # Potencia (**) ou ^
11    ('PLUS', r'\+'), # Soma
12    ('MINUS', r'-'), # Subtracao
13    ('MUL', r'\*'), # Multiplicacao
14    ('DIV', r'/'), # Divisao
15    ('ASSIGN', r('='), # Atribuicao
16    ('LPAREN', r'('), # (
17    ('RPAREN', r')'), # )
18    ('LBRACK', r '['), # [
19    ('RBRACK', r']'), # ]
20    ('LBRACE', r'{'), # {
21    ('RBRACE', r'}'), # }
22    ('SKIP', r'[\t\n]+'), # Pular espacos e quebras
    de linha
23    ('MISMATCH', r'.'), # Qualquer outro caractere
    invalido
24 ]
25
26
27 def tokenize(code):
28     """Transforma o texto bruto em uma lista de tokens."""
29     tok_regex = '|'.join(f'(?P<{name}>{regex})' for name,
    regex in TOKEN_SPECIFICATION)
30     tokens = []
31     for mo in re.finditer(tok_regex, code):
32         kind = mo.lastgroup
33         value = mo.group()
34         if kind == 'NUMBER':
35             tokens.append(('NUMBER', float(value)))
36         elif kind == 'IDENT':
37             tokens.append(('IDENT', value))
38         elif kind == 'SKIP':
```

```

39         continue
40     elif kind == 'MISMATCH':
41         raise SyntaxError(f'Caractere inesperado: {value}'
42     ')
43     else:
44         tokens.append((kind, value))
45     tokens.append(('EOF', None)) # Fim do arquivo/texto
46     return tokens

```

Listing 1: Analisador Léxico em Python